

Neural Style Transfer with VGG19

Scaler School of Technology | Deep Learning I | Week 8 — Session 2/5 | Post-Lecture Review & Practice



Key Takeaways

1 CNNs encode a hierarchy of representations

Shallow layers (conv1_1, conv2_1) detect local textures, edges, and color blobs. Deep layers (conv4_1, conv5_1) encode semantic objects and scene structure. Neural style transfer exploits this hierarchy: **content** is read from deep layers (conv4_2), while **style** is read from multiple shallow-to-deep layers.

2 Optimization targets the image, not the network

VGG19 weights are completely frozen. We use gradient descent (LBFGS optimizer) to minimize a composite loss by **modifying the pixel values** of the generated image. The generated image starts as a copy of the content image and is iteratively adjusted to also match the style statistics of the painting.

3 Gram matrices encode style as feature correlations

For a feature map $F^l \in \mathbb{R}^{C \times HW}$, the Gram matrix $G_{ij}^l = \frac{1}{CHW} \sum_k F_{ik}^l F_{jk}^l$ captures pairwise co-activations between feature channels. This encodes *which textures co-occur* while discarding spatial location — making it an ideal style descriptor.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1. Why does neural style transfer optimize the pixel values of the generated image rather than the network weights?
- Q2. If you want the output to look more like the content photo and less like the painting, which weight would you increase — α (content) or β (style)? Explain with reference to the loss function.
- Q3. What information does the Gram matrix capture that raw feature maps do not?
- Q4. Why are shallow convolutional layers used for style extraction but deep layers for content extraction?
- Q5. Total variation loss is added to the objective. What artifact does it prevent, and how does it work mathematically?

Coding Exercises

Work through these in a Colab notebook with GPU enabled.

Exercise 1: Feature Map Visualization across VGG19 Layers

Load pretrained VGG19. Pass a test image through the network and extract activations at conv1_1, conv2_1, conv3_1, conv4_1, and conv5_1. For each layer, visualize the first 8 feature maps as a grid using matplotlib. Write 1–2 sentences per layer describing what visual information it captures. *Expected output:* 5 grids (8 feature maps each) showing the progression from edge detection to semantic features.

Exercise 2: Gram Matrix Analysis and Shift Invariance

Compute Gram matrices for a style image at conv1_1, conv3_1, and conv5_1. Visualize each as a heatmap. Then apply a spatial shift (roll the image by 50 pixels) and recompute the Gram matrices. Compare the before/after Gram matrices using MSE. *Hypothesis:* the Gram matrix should be nearly identical after the shift, confirming style is spatially invariant. *Expected output:* 6 Gram heatmaps + MSE comparison table.

Exercise 3: Style Transfer on Your Own Image

Upload your own photo as the content image. Use one of the provided style images (Starry Night, Great Wave, or Picasso). Run 300 optimization iterations with LBFGS. Save the generated image at iterations 0, 50, 150, and 300. Create a 2×2 grid showing the stylization progress. *Expected output:* progression grid showing the style gradually transferring while preserving content structure.

Challenge Problem

Multi-Style Blending — Blend two style images simultaneously:

1. Load two style images (e.g., Starry Night + Picasso cubist)
2. Compute style loss as a *weighted sum* from both style images: $\mathcal{L}_{\text{style}} = w_1 \mathcal{L}_{\text{style1}} + w_2 \mathcal{L}_{\text{style2}}$
3. Run style transfer with weight combinations: [0.7, 0.3], [0.5, 0.5], [0.3, 0.7]
4. Display all three results side-by-side for comparison

Bonus: Preserve original content image colors by transferring only luminance (convert to LAB color space, apply style to L channel only, recombine with original A/B channels).

Further Reading

- **Original NST Paper:** Gatys, Ecker & Bethge (2016), “A Neural Algorithm of Artistic Style” — the paper that introduced Gram-matrix style transfer.
- **Fast Style Transfer:** Johnson, Alahi & Fei-Fei (2016), “Perceptual Losses for Real-Time Style Transfer” — feed-forward network for 1000× faster stylization.
- **AdaIN:** Huang & Belongie (2017), “Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization” — align feature statistics rather than Gram matrices.
- **VGG Architecture:** Simonyan & Zisserman (2015), “Very Deep CNNs for Large-Scale Image Recognition” — the backbone network used in NST.

Online Resources & Tools

- **PyTorch torchvision.models** — pretrained VGG19
- **Magenta Arbitrary Stylization** — TF Hub demo
- **TensorFlow NST Tutorial** — official walkthrough
- **PIL / Pillow** — image loading and manipulation
- **DeepDream Colab** — related CNN visualization
- **OpenCV** — advanced image processing

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about Gram matrices?** 3Blue1Brown linear algebra series Ep. 7 (matrix multiplication) — the Gram matrix $G = FF^T$ is simply a self-outer-product measuring feature correlations.
- **Unsure about LBFGS optimizer?** It is a quasi-Newton second-order optimizer. Unlike Adam, it uses curvature information and requires full-batch evaluation — ideal for image pixel optimization.
- **Forgot what feature maps look like?** Revisit Week 4 CNN lecture — feature maps are the output of convolutional filters, one per channel per layer.
- **Why no BatchNorm in VGG19?** The original architecture predates BatchNorm (2015). Modern VGG-BN variants add it, but the pretrained weights used for NST are from the original.

Answers — Test Your Understanding

- A1.** VGG19 is used as a *fixed feature extractor*. Modifying its weights would destroy the pre-trained representations that encode both content and style. Instead, the generated image is the variable being optimised — it starts as a copy of the content image and pixel values are iteratively adjusted.
- A2.** Increase α (content weight). The total loss is $\mathcal{L} = \alpha\mathcal{L}_{\text{content}} + \beta\mathcal{L}_{\text{style}} + \gamma\mathcal{L}_{\text{TV}}$. A higher α pulls the optimization toward preserving content features at the cost of style deviation.
- A3.** The Gram matrix captures *pairwise co-activations between feature channels* — which textures tend to appear together. Raw feature maps preserve spatial location, but Gram matrices collapse spatial dimensions via the sum over k (spatial positions), making style descriptors location-invariant.
- A4.** Shallow layers have small receptive fields (3×3 to $\sim 20 \times 20$ pixels) and detect local patterns like brush strokes, colors, and textures — exactly what defines artistic style. Deep layers have receptive fields covering most of the image and encode semantic objects (“this is a sky”) — exactly what defines image content.
- A5.** Without TV loss, optimizing style and content independently introduces high-frequency noise (checkerboard artifacts) because adjacent pixels are adjusted independently. TV loss penalizes $\sum_{i,j} (|I_{i,j+1} - I_{i,j}|^2 + |I_{i+1,j} - I_{i,j}|^2)$, enforcing local smoothness by penalising sudden pixel changes.

Deep Dive: Feature Hierarchies in VGG19

Layer	Receptive Field	What It Detects	Role in NST
conv1_1	3×3	Edges, color gradients	Style (fine textures)
conv2_1	7×7	Corners, simple patterns	Style (textures)
conv3_1	15×15	Textures, simple shapes	Style (patterns)
conv4_1	31×31	Object parts	Style (high-level)
conv4_2	31×31	Object parts (same block)	Content
conv5_1	63×63	Semantic objects	Style (semantics)

The choice of conv4_2 for content is a design decision in the original paper: deep enough to capture semantic structure, but not so deep that fine spatial detail is lost.

Deep Dive: The Gram Matrix — Formal Definition

Let $F^l \in \mathbb{R}^{C_l \times N_l}$ where C_l = number of channels and $N_l = H_l \times W_l$ = number of spatial positions. The **normalized Gram matrix** is:

$$G_{ij}^l = \frac{1}{C_l N_l} \sum_{k=1}^{N_l} F_{ik}^l \cdot F_{jk}^l$$

G_{ij}^l measures how strongly feature i and feature j co-activate across all spatial positions. The normalization by $C_l N_l$ is critical: without it, gradient magnitudes scale with layer size, making style weights layer-dependent.

The **style loss** at layer l compares Gram matrices of the style image $\mathbf{a}$ and generated image $\mathbf{x}$:

$$\mathcal{L}_{\text{style}}^l = \frac{1}{4} \left\| G^l[\mathbf{x}] - G^l[\mathbf{a}] \right\|_F^2$$

Deep Dive: LBFGS vs Adam for Image Optimization

Adam (first-order):

- Uses only gradient information
- Supports mini-batches
- Robust, general-purpose
- Slower convergence for smooth objectives
- ~500–1000 iterations needed

LBFGS (quasi-Newton):

- Approximates second-order (curvature) info
- Requires full-batch evaluation per step
- Much faster for smooth, low-dim problems
- Standard choice for NST image optimization
- ~300 iterations typically sufficient

LBFGS is preferred for NST because the image pixel space is smooth and the entire “dataset” is a single image (no mini-batching needed). For training neural network weights on large datasets, Adam wins.

Systematic Style Transfer Experiments

Record results in a table. Each experiment changes one variable while holding others constant:

1. **Content weight sweep:** Fix $\beta = 10^6$, vary $\alpha \in \{0.1, 1, 10, 100\}$. Observe the transition from “all style, no structure” to “original photo with slight texture”.
2. **Style weight sweep:** Fix $\alpha = 1$, vary $\beta \in \{10^3, 10^4, 10^5, 10^6\}$. At what value does style become overpowering?
3. **Number of style layers:** Use only conv1_1 (1 layer), then add layers one by one up to 5. How does adding deeper style layers change the result?
4. **Initialization comparison:** Initialize with (a) a copy of the content image, (b) pure Gaussian noise. Run 300 iterations each. Does initialization affect the final result?

Video Style Transfer (Advanced)

Apply NST frame-by-frame to create stylized video:

1. Extract frames from a short video clip using OpenCV: `cv2.VideoCapture`
2. Apply the standard NST pipeline to each frame independently
3. Reassemble frames with `cv2.VideoWriter`
4. Observe the problem: each frame is optimized independently, causing *temporal flickering*
5. Solution: add a temporal consistency loss: $\mathcal{L}_{\text{temp}} = \|I_t - \text{warp}(I_{t-1}, \phi_{t-1 \rightarrow t})\|^2$ where ϕ is the optical flow field

Neural Texture Synthesis

Run NST *without* a content image — optimize only for style loss. The result is a pure texture generated from the style image's statistics:

1. Initialize the generated image from pure Gaussian noise
2. Set $\alpha = 0$ (no content loss), $\beta = 1$ (style loss only)
3. Optimize for 500 iterations
4. Compare textures generated using only conv1_1 vs. all 5 style layers
5. *Observation:* shallow layers produce fine-grain textures; combining all layers produces rich, multi-scale textures

VGG19 Architecture Reference

Block	Layers	Output Size (512×512 input)	Parameters
Block 1	conv1_1, conv1_2, MaxPool	256 × 256 × 64	38K
Block 2	conv2_1, conv2_2, MaxPool	128 × 128 × 128	148K
Block 3	conv3_1–conv3_4, MaxPool	64 × 64 × 256	1.2M
Block 4	conv4_1–conv4_4, MaxPool	32 × 32 × 512	4.7M
Block 5	conv5_1–conv5_4, MaxPool	16 × 16 × 512	9.4M
FC Layers	fc6, fc7, fc8	4096, 4096, 1000	124M

Note: NST uses only the convolutional blocks (frozen). The FC layers are discarded.

NST Hyperparameter Guide

Parameter	Typical Range	Effect	How to Tune
content_weight α	1–100	Higher $\Rightarrow$ preserves content structure	Increase if output loses shape
style_weight β	10^4 – 10^6	Higher $\Rightarrow$ stronger style effect	Decrease if content disappears
tv_weight γ	10^{-4} – 10^{-2}	Higher $\Rightarrow$ smoother output	Increase if result is noisy
n_iterations	300–1000	More $\Rightarrow$ better convergence	Monitor loss plateau
image_size	256–512px	Larger $\Rightarrow$ finer detail, slower	512 on Colab GPU
Style layers	1–5	More layers $\Rightarrow$ richer style transfer	Start with all 5

Common Issues & Fixes

Symptom	Likely Cause	Fix
Output too noisy/grainy	TV weight too low	Increase γ by $10\times$
Content structure lost	Style weight too high	Reduce β by $10\times$
Unnatural colors	Style dominates color channels	Try color-preserving NST (match luminance only)
Optimization diverges	LR too high	Reduce LBFGS learning rate; try Adam instead
OOM error on Colab	Image size too large	Reduce to 256px; use gradient checkpointing
Very slow convergence	Initializing from noise	Initialize from content image instead

Hands-on notebook: `Session_2_Neural_Style_Transfer.ipynb`

Instructor: Priyansh Saxena